

Conception d'un systeme RAG

Assistant question-reponse sur les concepts de NLP/Machine Learning

Auteur : Mohamed Salem Ebnou Echvaghera Oubeid - C34613
Module NLP - Projet RAG (Master 1, Semestre 2, 2025-2026)
Rapport de projet - Retrieval-Augmented Generation

Domaine choisi : concepts fondamentaux du traitement automatique du langage naturel (NLP) et du machine learning, a partir d'un corpus d'articles Wikipedia.

Table des matieres

1. Introduction
2. Architecture du systeme
3. Donnees et preparation du corpus
4. Implementation
5. Methodologie d'evaluation
6. Resultats et visualisations
7. Discussion
8. Conclusion et perspectives
9. References

1. Introduction

Ce projet a été réalisé dans le cadre du module NLP du Master 1. Il consiste à concevoir, implémenter et évaluer un système complet de génération augmentée par récupération (Retrieval-Augmented Generation, RAG), capable de répondre à des questions en langage naturel en s'appuyant sur un corpus de documents et un grand modèle de langage (LLM).

Le domaine choisi est celui des **concepts fondamentaux du NLP et du machine learning** (transformers, embeddings, recherche d'information, métriques d'évaluation, etc.). Ce choix permet de constituer un corpus cohérent et de qualité à partir d'articles Wikipedia, tout en restant directement utile pour réviser les notions vues dans le module lui-même : le système peut être utilisé comme un assistant de révision répondant à des questions sur ces concepts.

L'objectif général du projet est de couvrir l'ensemble de la chaîne d'un système RAG : collecte et nettoyage des données, découpage en chunks, indexation vectorielle, pipeline de récupération et de génération, interface utilisateur, évaluation quantitative et visualisation des résultats. Chaque étape est implémentée dans un script Python dédié et documentée dans ce rapport.

Le rapport est organisé comme suit :

- L'**architecture** générale du système (section 2) ;
- Les **données** utilisées et leur préparation (section 3) ;
- Les choix d'**implémentation** (modèle d'embedding, base vectorielle, LLM, prompt, interface) (section 4) ;
- La **methodologie d'évaluation** (section 5) ;
- Les **resultats** obtenus, accompagnés de graphiques (section 6) ;
- Une **discussion** sur les forces et limites du système (section 7) ;
- La **conclusion** et les pistes d'amélioration (section 8).

2. Architecture du systeme

Le systeme suit l'architecture classique d'un pipeline RAG, divisee en deux phases : une phase d'**indexation hors ligne** (executee une seule fois, ou a chaque mise a jour du corpus) et une phase de **reponse en ligne** (executee a chaque question posee par l'utilisateur). La figure 1 illustre ce flux.

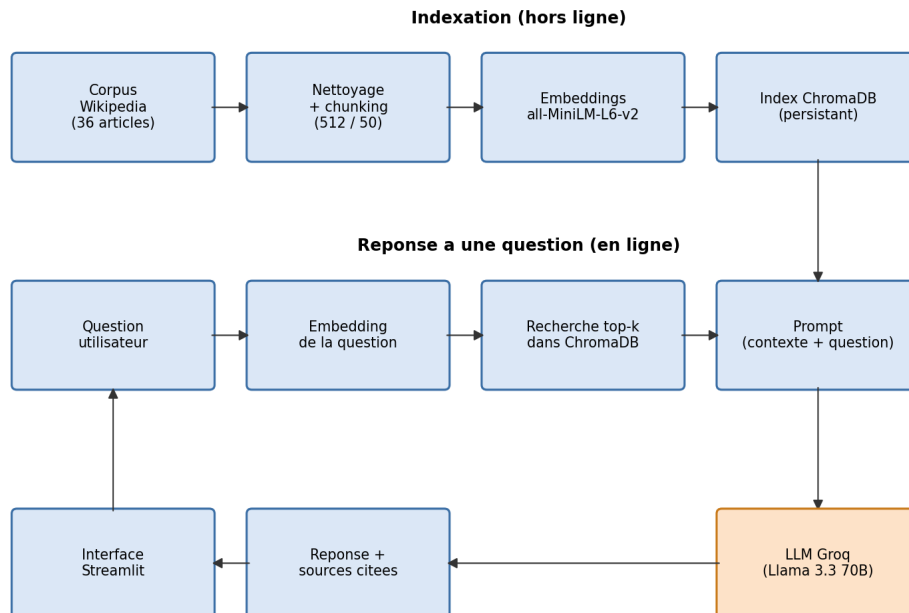


Figure 1 - Architecture generale du pipeline RAG.

2.1 Phase d'indexation (hors ligne)

- **Corpus** : 36 articles Wikipedia telecharges automatiquement par `collect_corpus.py` ;
- **Nettoyage et chunking** : suppression des pages d'homonymie et du contenu non pertinent (sections "See also", "References", etc.), puis decoupage en chunks de 512 caracteres avec un chevauchement de 50 caracteres (`src/prepare_data.py`) ;
- **Embeddings** : chaque chunk est vectorise avec le modele `all-MiniLM-L6-v2` (`sentence-transformers`) ;
- **Index vectoriel** : les vecteurs, textes et metadonnees (document source) sont stockes dans une collection ChromaDB persistante (`src/build_index.py`).

2.2 Phase de reponse (en ligne)

- La question de l'utilisateur est encodee avec le meme modele d'embedding ;
- ChromaDB renvoie les top-3 chunks les plus proches (similarite cosinus) ;
- Ces chunks sont inseres dans un template de prompt avec la question ;
- Le prompt est envoye au LLM via l'API Groq (modele `llama-3.3-70b-versatile`), qui genere une reponse ancree dans le contexte ;
- La reponse, les sources utilisees et des metriques (temps de reponse, fidelite) sont affichees dans l'interface Streamlit.

3. Donnees et preparation du corpus

Le corpus est constitue de 36 articles Wikipedia en anglais, couvrant les principaux concepts du NLP et du machine learning abordes dans le module (modeles de langage, embeddings, architectures de reseaux de neurones, metriques d'evaluation, etc.). Les articles sont telecharges automatiquement avec la bibliotheque wikipedia et sauvegardes en texte brut dans `data/raw/`.

3.1 Nettoyage

Deux articles se sont reveles etre des pages d'homonymie ("BM25" et "Tokenization") et ne contenaient pas de contenu exploitable ; ils ont ete ecartes, le sujet de la tokenisation etant deja couvert par l'article *Tokenization (lexical analysis)*. Le corpus final compte donc **34 documents**, ce qui respecte la contrainte minimale de 30 documents. Pour chaque document conserve, les sections de fin generiques et non informatives ("See also", "References", "Notes", "External links") sont supprimees afin de ne garder que le contenu encyclopedique pertinent.

3.2 Decoupage en chunks

Chaque document nettoye est decoupe en chunks de **512 caracteres** avec un chevauchement de **50 caracteres** entre chunks consecutifs. Ce chevauchement permet d'eviter qu'une information importante soit coupee entre deux chunks et perdue lors de la recherche. Ce decoupage produit **1443 chunks**, stockes avec leur identifiant, leur document source et leur texte dans `data/processed/chunks.jsonl` (format JSON Lines).

3.3 Documents du corpus

Le tableau suivant liste les 34 articles Wikipedia utilises (titres originaux en anglais) :

Attention (machine learning)	Question answering (computing)
BERT (language model)	ROUGE (metric)
BLEU	Recurrent neural network
Bag-of-words model	Retrieval-augmented generation
Cosine similarity	Sentence embedding
FastText	Sentiment analysis
Fine-tuning (deep learning)	Seq2seq
GPT (language model)	Stemming
GloVe (machine learning)	Stop word
Hallucination (artificial intelligence)	TF-IDF
Information retrieval	Text classification
LangChain	Text segmentation
Lemmatization	Tokenization (lexical analysis)
Long short-term memory	Transfer learning
N-gram	Transformer (machine learning)
Named-entity recognition	Vector database
Perplexity (natural language processing)	Word2vec

4. Implementation

4.1 Choix techniques

Les choix techniques ont été centralisés dans `config.py` afin d'éviter toute duplication de constantes (taille de chunk, chevauchement, modèle d'embedding, top-k, modèle LLM).

Composant	Choix	Justification
Langage	Python 3.13	Ecosystème NLP/ML mature (sentence-transformers, ChromaDB, Streamlit).
Embeddings	all-MiniLM-L6-v2	Modèle compact (~80 Mo), rapide en CPU, bonnes performances pour la recherche sémantique.
Base vectorielle	ChromaDB (persistant)	Simple à intégrer, ne nécessite pas de serveur externe, persistance locale sur disque.
LLM	Groq - Llama-3.3-70b-versatile	API gratuite et très rapide (inférence LPU), modèle open-weight performant.
Interface	Streamlit	Permet de créer rapidement une interface web interactive en Python pur, avec onglets et boutons.
Decoupage	512 caractères / chevauchement de 50	Compromis entre granularité (chunks suffisamment spécifiques) et contexte (chunks suffisamment longs).
Top-k	3	Nombre de chunks fournis au LLM ; suffisant pour couvrir la réponse sans diluer le contenu.

4.2 Pipeline de récupération et de génération

Le module `src/rag_pipeline.py` implémente les fonctions centrales du système : `retrieve()` (encode la question et interroge ChromaDB), `build_prompt()` (assemble le contexte récupéré et la question dans un template) et `generate_answer()` (appelle le LLM Groq et retourne la réponse ainsi que les chunks sources). Le template de prompt demande explicitement au modèle de répondre uniquement à partir du contexte fourni, et d'indiquer s'il ne connaît pas la réponse - ce qui limite les hallucinations.

Une version interactive en ligne de commande (`python -m src.rag_pipeline`) permet de poser des questions et d'afficher la réponse ainsi que les documents sources utilisés.

4.3 Interface Streamlit

L'interface (`app.py`) est organisée en trois onglets :

- **Chat** : interface conversationnelle pour poser des questions, avec affichage du temps de réponse, du nombre de chunks utilisés, d'un score de fidélité, et des extraits sources avec leur score de similarité ;
- **Evaluation** : permet de charger un jeu de questions (JSON/CSV), de lancer l'évaluation automatique, d'afficher les métriques agrégées, des graphiques interactifs (Plotly) et d'exporter les résultats en CSV ou en PDF ;
- **Gestion du corpus** : affiche des statistiques sur le corpus (nombre de documents, de chunks, taille moyenne), permet d'ajouter de nouveaux documents (PDF/TXT), de sélectionner les sources à indexer et de reconstruire l'index vectoriel.

La barre latérale permet de choisir le modèle LLM Groq, d'ajuster le nombre de chunks récupérés (k) et le seuil de similarité, et d'afficher l'état de l'index vectoriel et de la clé API.

5. Methodologie d'evaluation

Un jeu de **34 questions** a ete constitue manuellement (`evaluation/eval_questions.json`), couvrant l'ensemble des 34 documents du corpus - une question par concept, et quelques questions associees a deux documents lorsque les notions sont etroitement liees (par exemple lemmatisation/stemmatisation, ou LSTM/RNN). Cela depasse le minimum de 20 questions demande par le sujet.

Pour chaque question, le script `evaluation/evaluate.py` execute le pipeline complet (recuperation + generation) et calcule quatre metriques :

- **Precision@3** : proportion des 3 chunks recuperes qui appartiennent effectivement au(x) document(s) attendu(s) pour la question ;
- **Recall@3** : proportion des documents attendus qui figurent parmi les chunks recuperes (indique si l'information necessaire a ete retrouvee) ;
- **Fidelite (faithfulness)** : proportion des mots de la reponse generee qui apparaissent egalement dans le contexte recupere - une mesure simple, sans dependance externe, de l'ancrage de la reponse dans les sources (absence d'hallucination) ;
- **Temps de reponse** : duree totale (en secondes) entre l'envoi de la question et la reception de la reponse du LLM, incluant l'encodage de la question, la recherche vectorielle et l'appel a l'API Groq.

Les resultats details (une ligne par question) sont sauvegardes dans `evaluation/results.csv`, puis transformes en graphiques par `visualizations/plot_results.py` (section suivante). La meme logique d'evaluation est disponible de maniere interactive dans l'onglet "Evaluation" de l'interface Streamlit, qui permet de charger n'importe quel autre jeu de questions.

6. Resultats et visualisations

L'evaluation a ete executee sur les 34 questions du jeu de test. Le tableau ci-dessous resume les moyennes obtenues :

Metrique	Valeur moyenne
Precision@3	0.922
Recall@3	0.985
Fidelite (faithfulness)	0.887
Temps de reponse (s)	0.811

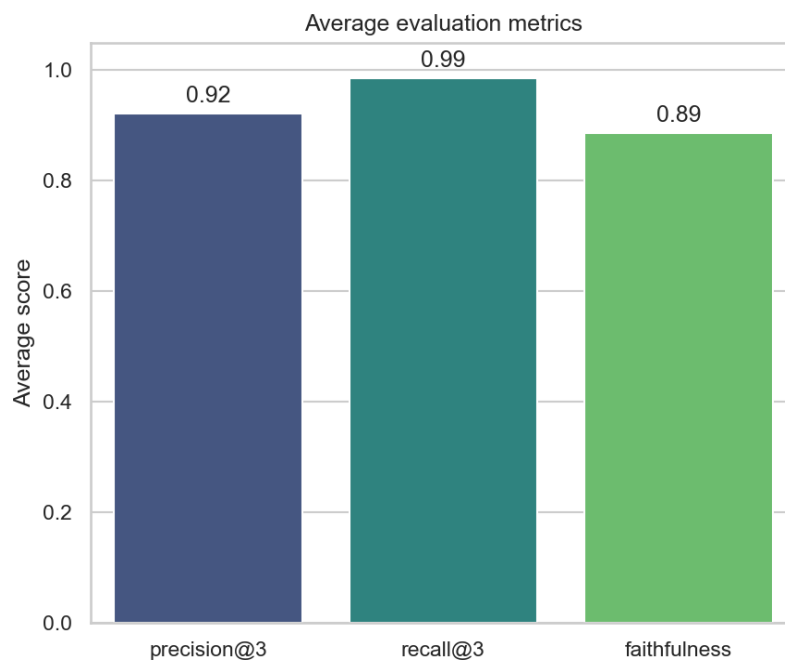


Figure 2 - Moyennes des metriques d'evaluation sur les 34 questions.

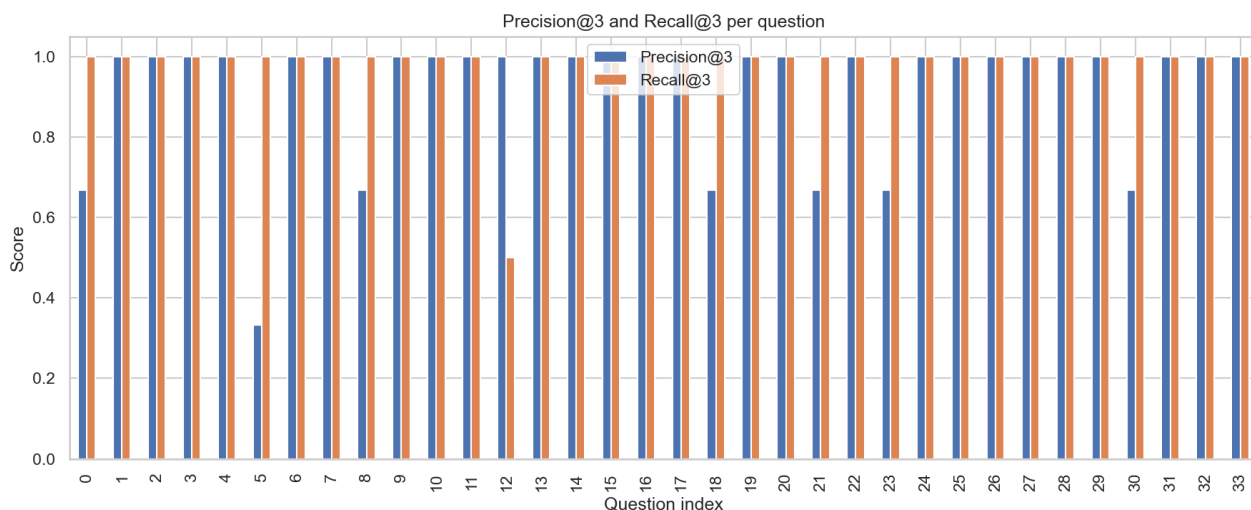


Figure 3 - Precision@3 et Recall@3 pour chacune des 34 questions. La grande majorité des questions atteignent une précision et un rappel de 1.0 ; quelques questions (par exemple celles couvrant un sujet aussi traité par un document voisin, comme lemmatisation/stematisation) présentent une précision ou un rappel plus bas car les chunks recuperes proviennent en partie d'un document lie mais different de celui attendu.

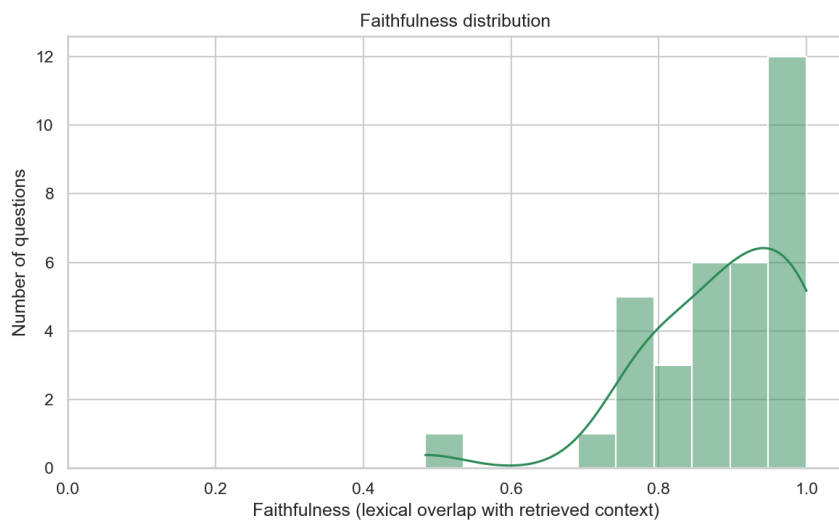
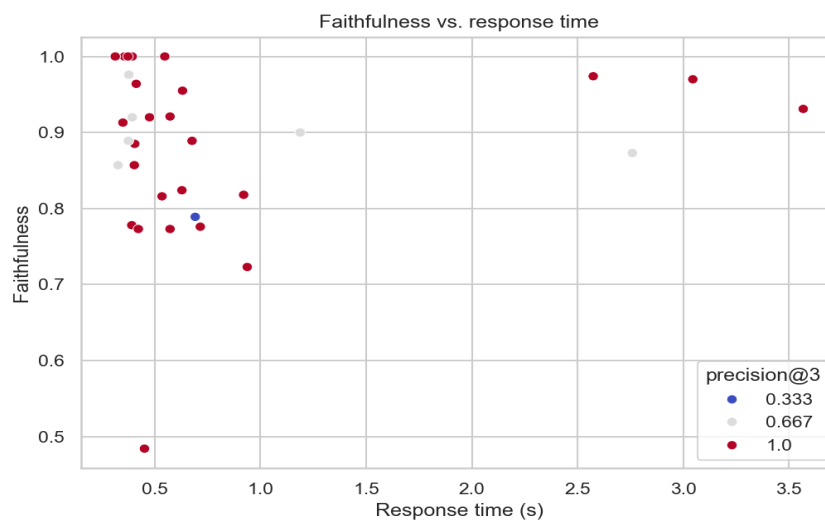
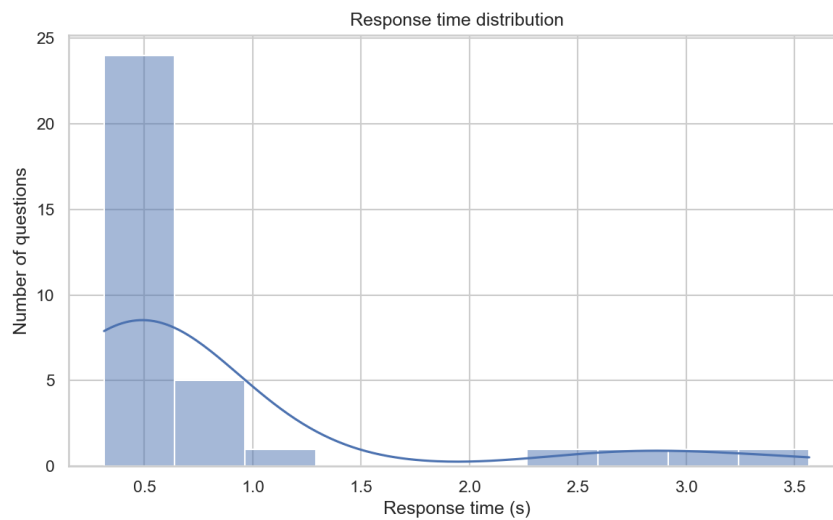


Figure 4 - Distribution du score de fidelite (recouvrement lexical entre la reponse generee et le contexte recupere). La plupart des reponses ont un score superieur a 0.8, ce qui indique qu'elles restent bien ancrees dans les passages recuperes.



7. Discussion

7.1 Points forts

- Le pipeline atteint une **precision@3 moyenne de 0.92** et un **recall@3 moyen de 0.99** sur le jeu de 34 questions, ce qui montre que l'index vectoriel base sur all-MiniLM-L6-v2 retrouve quasi systematiquement le bon document source, meme pour des formulations differentes de celles du texte original.
- La **fidelite moyenne de 0.89** indique que les reponses generees par le LLM restent tres largement ancrees dans le contexte recupere, grace au prompt qui impose de repondre uniquement a partir du contexte fourni.
- Le **temps de reponse moyen de 0.81 seconde** rend l'assistant utilisable de maniere interactive, l'API Groq etant tres rapide meme pour un modele de 70 milliards de parametres.
- L'interface Streamlit centralise les trois besoins du projet (chat, evaluation, gestion du corpus), ce qui facilite a la fois la demonstration et la maintenance du systeme.

7.2 Limites observees

- Quelques questions obtiennent une precision plus faible (0.33 a 0.67) lorsque le sujet recoupe un document voisin (ex. "FastText" vs "Word2vec", "sentence embedding" vs "sentence-transformers"/"BERT") : les chunks les plus proches semantiquement n'appartiennent pas toujours au document considere comme "attendu", bien que le contenu reste pertinent.
- La metrique de fidelite utilisee (recouvrement lexical) est une approximation simple : elle ne detecte pas les reformulations correctes (synonymes) ni les hallucinations subtiles qui reutiliseraient le vocabulaire du contexte de maniere incorrecte. Une evaluation par un second LLM (LLM-as-judge) serait plus precise mais plus couteuse et plus lente.
- Le corpus etant issu de Wikipedia (anglais), le systeme repond en anglais et est limite aux connaissances disponibles au moment du telechargement des articles ; il ne reflete pas les developpements les plus recents du domaine.
- La recherche vectorielle ne tient pas compte du recoupement entre articles tres proches (par exemple les familles de modeles de langage), ce qui peut amener le retriever a melanger des chunks issus de documents differents mais traitant du meme sous-theme.

8. Conclusion et perspectives

Ce projet a permis de mettre en oeuvre l'ensemble de la chaine d'un systeme RAG, depuis la collecte et le nettoyage d'un corpus jusqu'a une interface utilisateur complete et une evaluation quantitative outillee. Les resultats obtenus (precision@3 = 0.92, recall@3 = 0.99, fidelite = 0.89, temps de reponse moyen = 0.81 s) montrent que le systeme repond de maniere pertinente, rapide et bien ancree dans les sources sur l'ensemble du corpus de concepts NLP/ML choisi.

Plusieurs pistes d'amelioration ont ete identifiees pour des travaux futurs :

- Remplacer la metrique de fidelite par une approche "LLM-as-judge" pour detecter des hallucinations plus subtiles ;
- Etendre le corpus avec des sources plus recentes ou multilingues ;
- Ajouter un re-ranking des chunks recuperes (par exemple avec un modele cross-encoder) pour ameliorer la precision sur les sujets qui se recoupent ;
- Permettre la citation des sources directement dans le texte de la reponse generee.

9. References

- Wikipedia, articles divers sur les concepts de NLP et de machine learning (en.wikipedia.org), consultes en juin 2026.
- Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP. (modele all-MiniLM-L6-v2)
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Documentation ChromaDB - <https://docs.trychroma.com/>
- Documentation Groq API - <https://console.groq.com/docs/>
- Documentation Streamlit - <https://docs.streamlit.io/>
- Documentation sentence-transformers - <https://www.sbert.net/>